

Make a mini portfolio website along with GitHub Copilot

Building a Developer Portfolio

Learning Objectives

By the end of this lesson, you should be able to:

- Use comments to give GitHub Copilot context about a small web-development task.
- Build a simple developer portfolio using **HTML, CSS, and JavaScript** with Copilot assistance.
- Refine Copilot prompts when the initial suggestion does not match the requirement.
- Use Copilot to generate and modify HTML elements, images, styles, and JavaScript behavior.
- Implement a simple **light/dark mode toggle**.
- Review and adjust Copilot-generated code rather than accepting every suggestion.
- Separate CSS from the HTML document into an external stylesheet.

This exercise is intentionally exploratory: Copilot may generate different code even when students use the same prompts. The objective is therefore to practice **guiding, reviewing, refining, and testing** AI-generated code rather than reproducing one exact solution.

1. The Development Approach

In this lab, we will build a small developer portfolio containing:

- a profile image;
- developer name;
- professional title;
- a horizontal list of technical skills;
- skill icons;
- a light/dark mode toggle.



Rather than manually implementing every element, we will first describe what we want and allow Copilot to propose portions of the implementation.

The overall workflow is:

Define requirement



Describe it with comments/prompts



Copilot proposes code



Review the suggestion



Accept / Modify / Refine



Run in browser



Inspect result



Repeat

NOTE — Expect Different Results

GitHub Copilot's output is not deterministic. Your HTML, CSS, class names, or JavaScript may differ from the examples shown in class even when you use similar prompts. Focus on whether the generated solution satisfies the requirements.

2. Step 1 — Create the Project

Create a new project folder and add two files:

```
portfolio/  
├── index.html  
└── style.css
```

We will use:

- index.html for the page structure;
- style.css for presentation and layout.

JavaScript for the light/dark mode behavior will initially be added as part of the page implementation.

3. Step 2 — Give Copilot High-Level Context

Before generating the page, describe what you intend to build using comments in index.html.

For example:

```
<!-- Create a portfolio website for me -->  
  
<!-- Create a div with my image and make the image circular -->  
  
<!-- Display my name in an h1 element -->  
  
<!-- Display my title, Developer, on the next line as a  
paragraph -->  
  
<!-- Display my skills in a horizontal list.  
Show an icon and the skill name.  
Skills: HTML, CSS, JavaScript, Bootstrap -->
```

These comments provide Copilot with a lightweight specification.

Notice that we are describing **what the page should contain** before deciding exactly how every element should be implemented.

TIP — Establish Context Before Asking for Details

Copilot generally has a better chance of producing relevant suggestions when the surrounding file already communicates the purpose and structure of the application.

4. Step 3 — Create the HTML Structure

Begin creating the HTML5 document.

A basic structure is:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
    <title>Developer Portfolio</title>
  </head>

  <body>

  </body>

</html>
```

As you type, Copilot may use your earlier comments to suggest:

- page containers;
- elements;
- headings;
- paragraphs;
- skill lists.

For example:

```
<div class="profile">
  

  <h1>Reza Mirsalari</h1>

  <p>Developer</p>
```

`</div>`

The exact generated structure may differ.

Review Before Accepting

Check that the generated HTML corresponds to your requirements:

Profile image ✓

Name ✓

Developer title ✓

Skills ✓

If Copilot starts suggesting unrelated sections that were not requested, reject those suggestions.

PITFALL — Letting Copilot Expand the Scope

Copilot may predict additional portfolio features. Do not accept extra functionality simply because it was suggested. Keep the implementation aligned with the current requirements.

5. Step 4 — Use the Correct Profile Image

Copilot may initially generate an image URL or placeholder that is not the image you want.

Replace it with the correct image URL:

```

```

Save the file and refresh the browser.

Verify that the correct image appears.

This demonstrates an important AI-assisted development pattern:

Copilot generates starting point

↓

Developer identifies incorrect detail

↓

Developer supplies correct information

↓

Result improves

Copilot does not necessarily know application-specific data unless you provide it.

6. Step 5 — Add Skill Icons

The portfolio should display skills such as:

HTML CSS JavaScript Bootstrap

Initially, Copilot may suggest an icon library such as Font Awesome.

However, suppose our requirement is specifically to use **Icons8 images**.

Instead of accepting the first implementation, refine the prompt:

```
<!-- Create a horizontal list to showcase my skills.  
      Use Icons8 images as icons.  
      Display the name of each skill.  
      Skills: HTML, CSS, JavaScript, Bootstrap -->
```

This additional detail reduces ambiguity.

The generated structure may conceptually resemble:

```
<ul class="skills">  
  <li>  
      
    <span>HTML</span>  
  </li>  
  
  <li>  
      
    <span>CSS</span>  
  </li>  
  
  <li>  
      
    <span>JavaScript</span>  
  </li>  
  
  <li>  
      
    <span>Bootstrap</span>  
  </li>  
</ul>
```

Key Lesson: Prompt Refinement

The progression is:

"Use icons"

↓

Too general

↓

"Use Icons8 images"

↓

More specific requirement

↓

More relevant implementation

TIP — Correct Copilot Through the Prompt

When the generated implementation is close but does not satisfy a specific requirement, you do not necessarily need to rewrite everything manually. Refine the prompt and explicitly identify what should change.

7. Step 6 — Fix Oversized Skill Images

After refreshing the browser, you may discover that the generated skill images are much too large.

This is a visual defect that is easy to identify.

Select the relevant code and ask Copilot something similar to:

Fix the image width and height. Make the size 50 pixels.

The resulting CSS may resemble:

```
.skills img {  
    width: 50px;  
    height: 50px;  
}
```

What does this do?

width: 50px;
sets the image width.

height: 50px;
sets the image height.

Refresh the browser and verify the result.

NOTE — Browser Feedback Is Part of the Development Cycle

A generated solution can be syntactically correct while still producing a poor visual result. For front-end development, repeatedly inspect the rendered page and refine the implementation.

8. Step 7 — Add a Light/Dark Mode Control

We will now extend the portfolio with interactive behavior.

The requirement is:

- place a light/dark mode control near the top-right;
- use a checkbox to represent the toggle;
- display sun and moon images;
- initially display the page in light mode.

Describe this requirement to Copilot:

```
<!-- Create a div containing a checkbox for dark mode
      and two images representing light mode and dark mode -->
```

Copilot may generate something conceptually similar to:

```
<div class="container-dark">
  <input type="checkbox" id="darkMode">

  <label for="darkMode">
    
    
  </label>
</div>
```

9. Step 8 — Review Generated Class Names

In the source example, Copilot initially reused an existing generic container class.

That can create unintended styling interactions.

Instead of keeping:

```
<div class="container">
```

use a more specific class such as:

```
<div class="container-dark">
```

This distinguishes the dark-mode control from the main page container.

WARNING — Inspect Generated Selectors

Reusing a CSS class can unintentionally apply existing styles to a new component. Check generated class and id values before accepting them.

10. Step 9 — Style the Toggle

Move to style.css.

Copilot may assist with styling the dark-mode container and checkbox.

For example:

```
.container-dark {  
  display: flex;  
  align-items: center;  
}  
  
.container-dark input {  
  display: none;  
}  
  
.container-dark label {  
  display: flex;  
  align-items: center;  
  cursor: pointer;  
}
```

Notice the selector:

.container-dark input

rather than simply:

input

The first selector limits the rule to inputs inside the dark-mode component.

This is preferable because we do not want to hide every <input> that might later appear on the page.

11. Step 10 — Control Which Icon Is Visible

Initially, the page should be in **light mode**.

Therefore, display the sun and hide the moon.

Create a reusable CSS class:

```
.hidden {  
  display: none;  
}
```

Then apply it to the moon icon initially:

```

```

```

```

The initial state becomes:

Light Mode

↓

Sun → Visible

Moon → Hidden

The `.hidden` class gives JavaScript a simple way to switch icon visibility later.

12. Step 11 — Generate the Toggle Behavior

We now need JavaScript to respond when the user changes the checkbox.

Copilot may suggest code that:

1. locates the checkbox;
2. locates the sun icon;
3. locates the moon icon;
4. locates the document body;
5. listens for the checkbox's change event;
6. changes the page appearance;
7. switches the visible icon.

Conceptually:

```
const toggle = document.getElementById("darkMode");
const sunIcon = document.getElementById("sunIcon");
const moonIcon = document.getElementById("moonIcon");

toggle.addEventListener("change", function () {

    // Change page appearance

    // Switch sun/moon visibility

});
```

The important line is:

```
toggle.addEventListener("change", function () {
```

This tells the browser:

Execute this function whenever the checkbox changes state.

13. Step 12 — Test the Toggle

Save the page and refresh the browser.

Click the sun/toggle control.

Verify that:

LIGHT MODE

↓ click

DARK MODE

and clicking again produces:

DARK MODE

↓ click

LIGHT MODE

Also verify that the icons change appropriately:

Light Mode → Sun

Dark Mode → Moon

If the visual state and icon state disagree, inspect the generated JavaScript before proceeding.

14. Step 13 — Update the Text Color

Changing only the background may make the text difficult to read.

The source exercise therefore also changes the text color when switching themes.

The intended behavior is:

Light Mode

Background → light

Text → dark

Dark Mode

Background → dark

Text → light

Copilot can assist in adding the appropriate `body.style.color` logic.

After making the change, test both modes again.

TIP — Test the Complete Visual State

Do not verify only whether the background changes. Check the background, text readability, and sun/moon icon together. A feature is complete only when its related elements remain consistent.

15. Step 14 — Move CSS to style.css

During rapid development, some styles may have been generated directly inside the HTML document.

For better organization, move those styles into:

`style.css`

Then ensure `index.html` references the stylesheet:

```
<link rel="stylesheet" href="style.css">
```

This belongs inside `<head>`:

```
<head>
```

```
...
```

```
  <link rel="stylesheet" href="style.css">
```

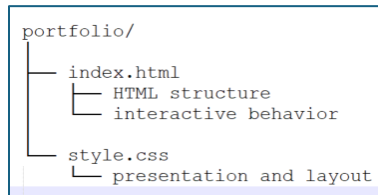
```
</head>
```

If Copilot does not suggest the link automatically, provide a direct comment:

```
<!-- Link style.css -->
```

and review the resulting suggestion.

The final organization should resemble:



16. Final Verification

Before considering the exercise complete, test each requirement.

Requirement	Verification
Profile image	Correct image displayed
Circular image	Image has intended appearance
Developer name	Visible as heading
Developer title	Visible below name
Skills	HTML, CSS, JavaScript, Bootstrap displayed
Skill icons	Correct images displayed
Icon size	Approximately 50 × 50 px as requested
Light mode	Correct background, text, and icon
Dark mode	Correct background, text, and icon
Toggle	Switches between both states
External CSS	style.css linked correctly

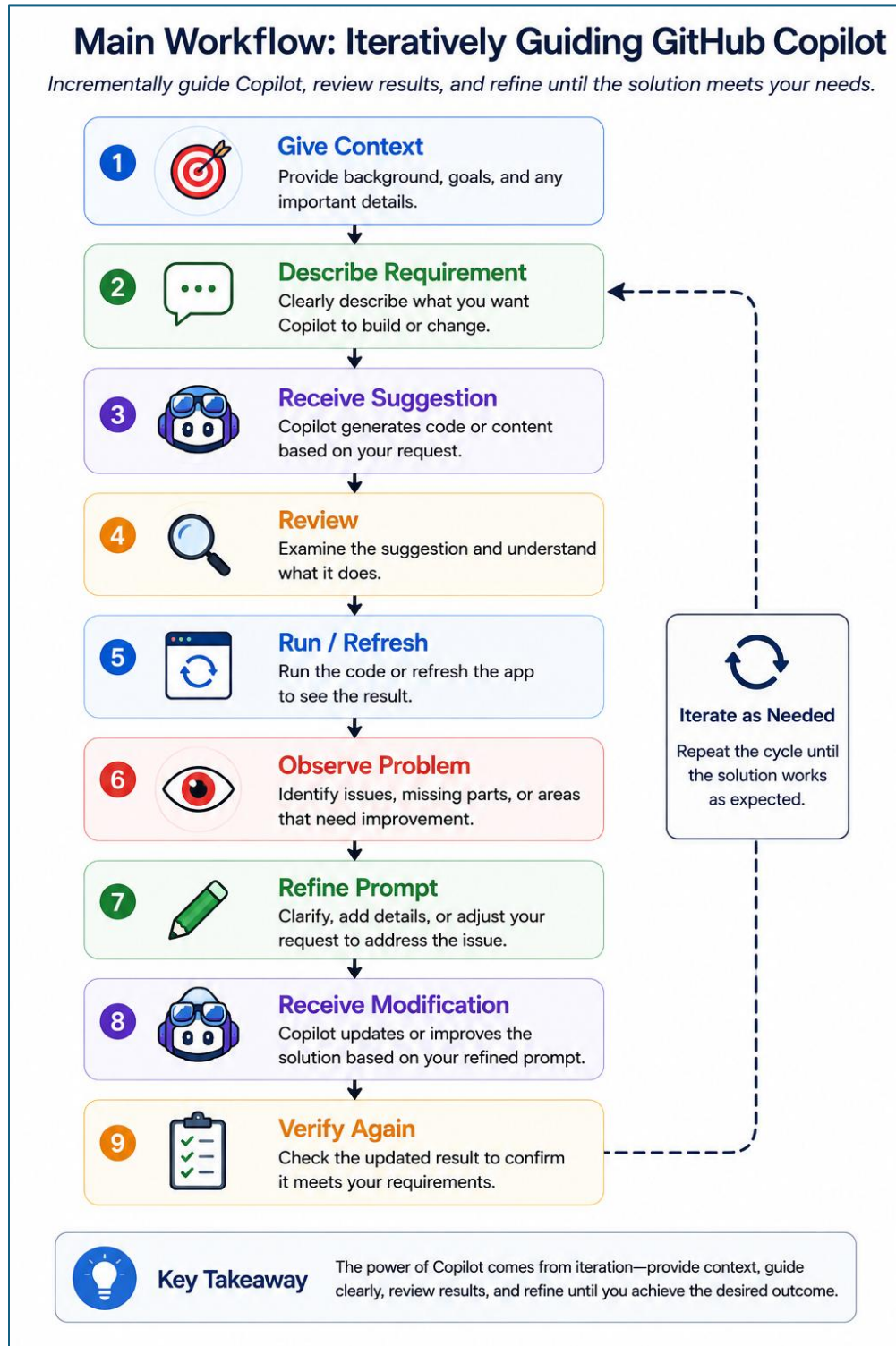
This verification step is particularly important because much of the implementation was generated incrementally.

17. What This Exercise Demonstrates

The main lesson is not that Copilot can generate a portfolio website. It is how a developer can **incrementally guide Copilot through a real development task**.

The main lesson is not that Copilot can generate a portfolio website. It is how a developer can **incrementally guide Copilot through a real development task**.

The workflow used throughout the exercise was:



18. Common Pitfalls

PITFALL — Accepting Unrequested Features

Copilot may suggest additional portfolio sections or functionality. Keep the implementation within the requirements of the current exercise.

PITFALL — Using Vague Prompts

"Add icons" leaves important decisions unspecified. "Use Icons8 images for HTML, CSS, JavaScript, and Bootstrap" communicates the requirement more precisely.

WARNING — Generated Code Still Requires Testing

A suggestion that looks correct in the editor may render incorrectly in the browser. Save, refresh, interact with the page, and verify the actual behavior.

NOTE — Your Result May Look Different

Copilot may generate different markup, CSS, or JavaScript from the examples. That is acceptable if your solution satisfies the same requirements and you understand the generated code.